🧠 Micro-Learner CLI: Project Roadmap

1. Project Overview

A terminal-based CLI application designed to turn idle developer time into structured micro-learning. Instead of doomscrolling or losing focus, the user can run this lightweight tool in their current terminal window to learn bite-sized, highly relevant concepts.

Key Design Philosophies:

- **Structured Progression:** Uses a generated "Curriculum Index" rather than random facts to ensure logical learning steps and prevent duplicate content.
- **Format Variety:** Alternates between deep-dive explanations and active micro-quizzes to keep cognitive engagement high.
- **Standalone CLI:** Runs natively in the current terminal window on demand (no forced Tmux pane orchestration).
- **Aesthetic UI:** Utilizes terminal styling (e.g., Python's `Rich` library) for a clean, distraction-free reading experience.

2. Technical Stack

- **Language:** Python 3
- **UI/Styling:** `Rich` (for Markdown rendering, progress bars, and colored text)
- **AI Integration:** Any LLM API (OpenAI, Anthropic, Gemini) or local via Ollama.
- **State Management:** Local JSON files (`~/.micro_learner/state.json` and `~/.micro_learner/syllabi/`).

3. Core Mechanics & The "Curriculum Index"

To avoid LLM amnesia and duplicated teachings, the app uses a **Curriculum Index** approach:

1. **Topic Initialization:** User runs the app and inputs a new topic (e.g., "Advanced Rust Traits").
2. **Syllabus Generation:** The app asks the LLM to generate a 15-20 step syllabus/outline on this topic and saves it as a JSON array.
3. **Step-by-Step Execution:** Each time the user requests a lesson, the app looks at the current index (e.g., Step 4: "Associated Types"), prompts the LLM to teach *specifically* that sub-topic, and increments the tracker to Step 5.

4. Implementation Phases

Phase 1: Foundation (The MVP)

Goal: Get the core loop working with local state and API calls.

- [ ] **Setup:** Initialize the Python project and necessary directories (`~/.micro_learner/`).

- [ ] **State Manager:** Write the logic to read/write a `current_state.json` file (tracking the active topic and current lesson index).
- [ ] **LLM Prompts:** * Write Prompt 1: Generate a JSON array of 15 sub-topics based on user input.
  - Write Prompt 2: Generate a 150-word lesson based on a specific sub-topic.
- [ ] **Basic CLI:** Create the loop: Check state -> If no topic, ask for one -> Fetch current lesson -> Display plain text -> Wait for `Enter` -> Increment index.

Phase 2: User Interface & Formatting

Goal: Make it beautiful and engaging.

- [ ] **Rich Markdown Integration:** Pass the LLM's output through `Rich` to render bold text, code blocks, and syntax highlighting.
- [ ] **Format Alternation:** Modify the prompt logic to randomly (or sequentially) ask the LLM to format the output as either a "Deep-Dive Explanation" or an "Active Micro-Quiz" (where the answer is hidden until the user presses a key).
- [ ] **The Progress Bar:** Implement the `Rich` progress bar at the bottom of the screen: `[▓▓▓

the LLM to reliably return the JSON syllabus without any conversational fluff.